LAB REPORT

Stack in Python

Push, Pop,Peek, Searching, balancedBrackets and int_to_binary_stack

Course	Data Structures and Algorithms Using Python
Lab No.	Lab 04
Student Name	Jiayu Yang （ID 20253801028）
Student ID	2025-AI-002
Submission Date	27 October 2026

1-1. Objective

The objective of this lab is to practice fundamental Stack operations in Python, including traversal, push, pop, peek, searching, balancedBrackets and int_to_binary_stack. The lab also demonstrates how these operations can be combined into a menu-driven Stack Management System.

1-2. Software / Environment

Programming Language	Python 3.x
Editor / IDE	Editor
Data Structure	Python Stack
File Name	20253801028_Stack_operations.py

2-1. Stack class definitions.

Stack Class program

```
class Stack:
    def __init__(self):
        self.stack = []
    def push(self, element):
        self.stack.append(element)
    def pop(self):
        if self.isEmpty():
            return "Stack is empty"
        return self.stack.pop()
    def peek(self):
        if self.isEmpty():
            return "Stack is empty"
        return self.stack[-1]
    def isEmpty(self):
        return len(self.stack) == 0
    def size(self):
        return len(self.stack)
    def clear(self):
        del self.stack[:]
    def dispBottomElement(self):
        print("BottomElement:", self.stack[-1])
    def getmax(self):
        max = 0
        for i in range(len(self.stack) - 1, -1, -1):
            if max < self.stack[0]:
                max  = self.stack[i]
        return max
    def getmin(self):
        if self.stack:
            min = self.stack[0]
            for i in range(len(self.stack) - 1, -1, -1):
                if min > self.stack[0]:
                    min  = self.stack[i]
            return min
        else:
            return 0xFFFF
    def getsum(self):
        sum = 0
```

```
        for i in range(len(self.stack) - 1, -1, -1):
            sum  += self.stack[i]
        return sum
    def traverse(self):
        for i in range(len(self.stack) - 1, -1, -1):
            print(self.stack[i])
    def search(self,value):
        for i in range(len(self.stack) - 1, -1, -1):
            if self.stack[i] == value:
                print("Found at index", i, "(0 = bottom)")
                return i
        print("Not found")
     def balancedBrackets(self):
        char = []
        print("Input 10 elements:")
        char = list(map(str, input().split()))
        tempchar = []
        for i in range(len(char) - 1, -1, -1):
            tempchar.append(char[i])
        print("Array:", tempchar)
        tempstack = []
        pairs = {')': '(', ']': '[', '}': '{'}
        for i in range(len(tempchar) - 1, -1, -1):
            temp = tempchar[i]
            if not isinstance(temp, str):
                continue
            ch = temp[0]
            if ch in "([{":
                tempstack.append(ch)
                #print("left to bottom:", tempchar[i])
            elif ch in ")]}":
                #print("right to bottom:", tempchar[i])
                if not tempstack or tempstack.pop() != pairs[ch]:
                    return False
        return not tempstack
    def int_to_binary_stack(self,num):
        if num <= 0:
            return "0"
        tempstack = []
        while num > 0:
            remainder = num % 2
            tempstack.append(remainder)
            num = num // 2
        binary_str = ""
        while tempstack:
            binary_str += str(tempstack.pop())
        return binary_str
    def reverseStack(self):
        tempstack = []
        for i in range(len(self.stack) - 1, -1, -1):
            tempstack.append(self.stack[i])
        return tempstack
```

Figure 1. Stack class definitions

2-2. Stack definitions、Stack() and MAIN MENU.

Requirement: Place all indented methods above inside Stack. Run the code below after the Node and Stack class definitions. Then, into main_menu, 7. into

Python code

```
    myStack = Stack()
    myStack.stack.append(10)
    myStack.stack.append(20)
    myStack.stack.append(30)
    print("Stack:", myStack.stack)
    print("Top:", myStack.stack[-1])
    myStack.push(40)
    print("myStack.stack", myStack.stack)
    myStack.push(50)
    print("myStack.push(50)->", myStack.stack)
    print("Top:", myStack.stack[-1])
    myStack.pop()
    print("myStack.pop->", myStack.stack)
    while True:
        print("\n===== STACK OPERATIONS =====")
        print("Jiayu Yang,ID:20253801028")
        print("1. Display  2. Push  3. Pop  4. Peek")
        print("5. Is Empty  6. Size 7. STACK MANAGEMENT SYSTEM 8. Exit")
        choice = input("Enter choice: ")
        if choice == "1":
            print("Top to bottom:", myStack.stack[::-1])
        elif choice == "2":
            myStack.stack.append(int(input("Value: ")))
            print("Element pushed")
        elif choice == "3":
            print("Popped:", myStack.pop() if myStack.stack else "Underflow")
        elif choice == "4":
            print("Top:", myStack.stack[-1] if myStack.stack else "Stack is empty")
        elif choice == "5":
            print("Empty:", not myStack.stack)
        elif choice == "6":
            print("Size:", len(myStack.stack))
        elif choice == "7":
            main_stack_management(myStack)
        elif choice == "8":
            print("Exit APP")
            break
        else:
             print("Invalid choice")
```

```
PS D:\AI\Data-Structure-in-python\lab> python .\20253801028_Stack_operations.py
Stack: [10, 20, 30]
Top: 30
myStack.stack [10, 20, 30, 40]
myStack.push(50)-> [10, 20, 30, 40, 50]
Top: 50
myStack.pop-> [10, 20, 30, 40]

===== STACK OPERATIONS =====
Jiayu Yang,ID:20253801028
1. Display  2. Push  3. Pop  4. Peek
5. Is Empty  6. Size 7. STACK MANAGEMENT SYSTEM 8. Exit
Enter choice:
```

Figure 2. Stack() and main_menu

Action:select task number 7 into STACK MANAGEMENT SYSTEM,..... .

2-3. Stack Management System

Stack Management System task1-15

Python code

```
def main_stack_management(stack):
    print("STACK MANAGEMENT SYSTEM")
    print(" 1. Display Stack (top to bottom)")
    print(" 2. Push Element")
    print(" 3. Pop Element")
    print(" 4. Peek Top Element")
    print(" 5. Check Whether Empty")
    print(" 6. Count Elements")
    print(" 7. Search Element")
    print(" 8. Clear Stack")
    print(" 9. Display Bottom Element")
    print(" 10. Find Maximum")
    print(" 11. Find Minimum")
    print(" 12. Calculate Sum")
    print(" 13. Reverse Stack into New Stack")
    print(" 14. Demonstrate Balanced Brackets")
    print(" 15. Decimal to Binary Conversion")
    print(" 16. Exit")
    while True:
        opt = input("\nPlease select task number: ")
        if opt == "1":
            print("Top to bottom:", stack.stack[::-1])
        elif opt == "2":
            val = (input("Enter element at top: "))
            stack.push(val)
        elif opt == "3":
            stack.pop()
        elif opt == "4":
            stack.peek()
        elif opt == "5":
            print("Empty:",stack.isEmpty())
        elif opt == "6":
            print("Size:", stack.size())
        elif opt == "7":
            val = int(input("Enter element at start: "))
            stack.search(val)
        elif opt == "8":
            stack.clear()
        elif opt == "9":
            stack.dispBottomElement()
        elif opt == "10":
            print("Max:", stack.getmax())
        elif opt == "11":
            print("Max:", stack.getmin())
        elif opt == "12":
            print("Sum:", stack.getsum())
        elif opt == "13":
            print("reverse stack:",stack.reverseStack())
        elif opt == '14':
```

```
            print(f"Match: {'Balanced' if stack.balancedBrackets() else 'Not
balanced'}")
        elif opt == "15":
            val = int(input("Enter element at start: "))
            print("IntDectoBinary:",stack.int_to_binary_stack(val))
        elif opt == "16":
            print("Exit APP")
            break
        else:
            print("Invalid option, try again.")
```

```
Enter choice: 7
STACK MANAGEMENT SYSTEM
 1. Display Stack (top to bottom)
 2. Push Element
 3. Pop Element
 4. Peek Top Element
 5. Check Whether Empty
 6. Count Elements
 7. Search Element
 8. Clear Stack
 9. Display Bottom Element
 10. Find Maximum
 11. Find Minimum
 12. Calculate Sum
 13. Reverse Stack into New Stack
 14. Demonstrate Balanced Brackets
 15. Decimal to Binary Conversion
 16. Exit

Please select task number: 1
Top to bottom: [40, 30, 20, 10]

Please select task number: 2
Enter element at top: 50

Please select task number: 4
Top peek: 50

Please select task number: 3

Please select task number: 5
Empty: False

Please select task number: 6
Size: 4

Please select task number: 7
Enter element at start: 40
Found at index 3 (0 = bottom)

Please select task number: 9
BottomElement: 40

Please select task number: 10
Max: 40

Please select task number: 11
Max: 10

Please select task number: 12
Sum: 100

Please select task number: 13
reverse stack: [40, 30, 20, 10]

Please select task number: 14
Input 10 elements:{ [ ( 6 + 4 ) * 5 ] }
Array: ['}', ']', '5', '*', ')', '4', '+', '6', '(', '[', '{']
Match: Balanced

Please select task number: 15
Enter element at start: 11
IntDectoBinary: 1011
```

Figure 2. Stack Management System task1-15

3. Test Cases

For the definitions Stack[10, 20, 30, 40]

1. FUNCTION Task2 push value 50, Stack[10, 20, 30, 40,50]

2. FUNCTION Task13 reverse stack, output [50, 40, 30,20, 10]
3. FUNCTION Task7 Search value 60, Not Found
4. FUNCTION Task7 Search value 30, search Found at index, 2, (0 = bottom)
5. FUNCTION Task14 Input `{ [ 5 * ( 6 + 3 ) ] }`, Balanced
6. FUNCTION Task14 Input `{ [ 5 *( 6 + 3 ] ) }`,Not Balanced
7. FUNCTION Task15 Input 11(Dec),1011(BIN)

```
Please select task number: 2
Enter element at top: 50
myStack.pop-> [10, 20, 30, 40, 50]

Please select task number: 13
reverse stack: [50, 40, 30, 20, 10]

Please select task number: 7
Enter element at start: 60
Not found

Please select task number: 7
Enter element at start: 30
Found at index 2 (0 = bottom)

Please select task number: 14
Input elements:{ [ 5 * ( 6 + 3 ) ] }
Array: ['}', ']', ')', '3', '+', '6', '(', '*', '5', '[', '{']
Match: Balanced

Please select task number: 14
Input elements:{ [ 5 * ( 6 + 3 ) ] }
Array: ['}', ']', ')', '3', '+', '6', '(', '*', '5', '[', '{']
Match: Balanced

Please select task number: 15
Enter element at start: 11
IntDectoBinary: 1011
```

Test	Operation	Input	Expected	Status

1	Task 2 push	50	[10,20,30,40,50]	Pass
2	Task13 reverse stack	30	[50, 40, 30,20, 10]	Pass
3	Task7 Search value	60	Not Found	Pass
4	Task7 Search value	30	search Found at index, 2, (0 = bottom)	Pass
5	Task14 Input	{ [5 * (6 + 3)] }	Balanced	Pass
6	Task14 Input	{ [5 * (6 + 3)] }	Not Balanced	Pass
7	Task15 D2B	11	1011	Pass

4. Lab tasks

attachment with 20253801028_Stack_operations.py and Section. 2 Screenshots showing program execution

5. Answers to the questions

Q1. What is a stack?

Answer: A stack is a linear data structure that follows a specific order for operations. It allows insertion and deletion of elements only at one end, known as the top. The other end is called the bottom and remains fixed. Because operations are restricted to one end, it is often described as a "Last-In, First-Out" (LIFO) structure.

Q2. What does LIFO mean? Give a real-world example.

Answer: LIFO stands for Last-In, First-Out. It means that the last element added to the stack is the first one to be removed.

- Real-world example: A stack of plates in a cafeteria. You place new plates on top of the pile (push). When you need a plate, you take the one from the very top (pop). You cannot easily take a plate from the bottom without removing all the plates above it first.

Q3. Which end of our Python list represents the stack top?

Answer: In Python, when using a standard list to implement a stack, the end of the list (the rightmost side) represents the stack top.

- list.append() adds an element to the end (Push).
- list.pop() removes an element from the end (Pop).
  This is efficient because adding/removing from the end of a dynamic array is an $O(1)$operation, whereas doing so at the beginning (index 0) would require shifting all other elements ($O(n)$).

Q4. How are push() and pop() different?

Answer: Push() Adds an element to the top of the stack. It increases the size of the stack by one. Pop() Removes and returns the element from the top of the stack. It decreases the size of the stack by one. Essentially, push is an insertion operation, while pop is a deletion/retrieval operation.

Q5. What is underflow?

Answer: Underflow occurs when you attempt to perform a pop or peek operation on an empty stack. Since there are no elements to remove or view, this is an error condition. In robust implementations, this should be handled by checking if the stack is empty before popping or by raising an exception.

Q6. When can overflow occur?

Answer: Overflow occurs when you attempt to push an element onto a stack that has reached its maximum capacity. This is relevant in implementations with a fixed size (like an array-based stack with a predefined limit). If the stack is full, no new elements can be added until space is freed by popping existing elements.

Q7. What is the time complexity of push, pop, and peek for a Python-list stack at the end?

Answer: For a Python list used as a stack (operating at the end of the list):

- Push (append): $O(1)$amortized time complexity.
- Pop (pop): $O(1)$time complexity.
- Peek (list[-1]): $O(1)$time complexity.
  These operations are constant time because they do not require shifting other elements in memory.

Q8. Does peek remove the top element?

Answer: No. The peek operation (often implemented as accessing the last index, e.g., stack[-1] in Python) only views or returns the value of the top element without modifying the stack. The element remains in the stack for future operations. In contrast, pop removes the element.

Q9. What is the difference between a stack and a queue?

Answer: Stack Follows LIFO (Last-In, First-Out). Elements are added and removed from the same end (the top). Queue Follows FIFO (First-In, First-Out). Elements are added at one end (rear/tail) and removed from the other end (front/head).Analogy: A stack is like a stack of plates; a queue is like a line of people waiting for a bus.

Q10. Can a stack be implemented using a linked list?

Answer: Yes. A stack can be efficiently implemented using a singly linked list. The head of the linked list acts as the top of the stack. Push: Insert a new node at the head. Pop: Remove the node from the head.Both operations are $O(1)$. This implementation avoids the "overflow" issue associated with fixed-size arrays, as memory is allocated dynamically (limited only by available system memory).

Q11. How are stacks used for function calls?

Answer: Computers use a Call Stack to manage function executions.When a function is called, a stack frame (containing local variables, parameters, and the return address) is pushed onto the call stack.When the function finishes execution, its stack frame is popped off the stack, and control returns to the previous function (using the stored return address).This ensures that functions return in the reverse order of their calls (LIFO), allowing for proper nesting and recursion.

Q12. Why are stacks useful for matching brackets?

Answer: Stacks are ideal for validating balanced parentheses (e.g., (), [], {}) because of the LIFO property. When an opening bracket is encountered, it is pushed onto the stack. When a closing bracket is encountered, the algorithm checks if it matches the bracket at the top of the stack (via peek or pop).If it matches, the opening bracket is popped. If it doesn't match or the stack is empty, the expression is invalid.At the end, if the stack is empty, all brackets were properly matched.

Q13. What is a postfix expression?

Answer: A postfix expression (also known as Reverse Polish Notation or RPN) is a mathematical notation where operators follow their operands.

Example: The infix expression A + B becomes A B + in postfix.

Example: (A + B) * C becomes A B + C *.

Postfix expressions do not require parentheses to define order of operations and are easily evaluated using a stack: operands are pushed, and when an operator is encountered, the required number of operands are popped, computed, and the result is pushed back.

Q14. Why does searching a stack take linear time in general?

Answer: Searching a stack takes $O(n)$ (linear) time because a stack is not designed for random access or searching. To find a specific element, you must potentially pop every element from the top down to the bottom until the target is found. Unlike arrays or hash tables, you cannot jump directly to an index or key.

Q15. How would you test a stack implementation for correctness?

Answer: To test a stack implementation, you should verify the following scenarios:

1. Initialization: Check if a new stack is empty.
2. Push: Add elements and verify the stack is no longer empty and the size increases.
3. Peek: Verify that peek returns the most recently pushed element without changing the stack size.
4. Pop: Verify that pop returns the most recently pushed element and reduces the stack size. Check that subsequent pops return elements in LIFO order.
5. Underflow Handling: Attempt to pop or peek from an empty stack and ensure it handles the error correctly (e.g., raises an exception or returns a specific error code).
6. Overflow Handling (if fixed-size): Attempt to push beyond capacity and ensure it handles the error correctly.
7. Mixed Operations: Perform a sequence of pushes and pops to ensure the state remains consistent